

Projeto de Software - CareConnect

UFU

2026

Sumário

1	Requisitos de Sistema	1
1.1	Escopo do Produto	1
1.2	Requisitos Funcionais	1
1.3	Requisitos Não-Funcionais	6
1.4	Protótipo	8
2	Projeto de Software	9
2.1	Diagramas de Interação	9
2.2	Diagramas de Classes	9
2.3	Persistência de Dados	10
2.4	Mapeamento de Ferramentas	10
2.4.1	Linguagem de Programação	10
2.4.2	Frameworks e Bibliotecas	10
2.4.3	Ferramentas de Desenvolvimento	11
2.4.4	Containerização e Automação	11
3	Versionamento	12
3.1	Introdução	12
3.2	Configuração do Ambiente	12
3.3	Versionamento Semântico (SemVer)	12
3.3.1	Estrutura da Versão (X.Y.Z)	12
3.4	O Modelo Git Flow	12
3.4.1	Branches Permanentes	12
3.4.2	Feature Branches (feature/*)	12
3.4.3	Release Branches (release/*)	13
3.4.4	Hotfix Branches (hotfix/*)	13
3.5	Exemplo:	13
3.6	Glossário de Comandos	14
3.6.1	Configuração Inicial	14
3.6.2	Operações Diárias	14
3.6.3	Boas Práticas de Commit	14

1 Requisitos de Sistema

O objetivo desta ERS (Especificação de Requisitos de Software) consiste em documentar os requisitos do software CareConnect a ser produzido. Este documento visa garantir que o cliente (instituição de saúde, cuidadores e pacientes idosos) e os desenvolvedores tenham um entendimento comum e inequívoco de todas as funcionalidades, capacidades e restrições do software, servindo como base para o design, desenvolvimento, testes e validação do produto final.

Este documento é destinado a desenvolvedores, testadores, gestores de projeto e demais stakeholders do projeto CareConnect.

1.1 Escopo do Produto

O sistema CareConnect tem como propósito **facilitar o acompanhamento e adesão à medicação por parte de idosos**, permitindo que cuidadores e familiares monitorem remotamente o tratamento e recebam alertas em caso de não adesão. O sistema integra dados de prescrição digital (ex.: Memed) e oferece lembretes via notificações push, SMS e ligações telefônicas.

Dentre os principais objetivos destacam-se:

- Permitir o cadastro e gestão de pacientes idosos e seus cuidadores
- Integrar prescrições médicas provenientes de plataformas digitais
- Enviar lembretes de medicação nos horários programados
- Registrar a tomada ou perda de dose dos medicamentos
- Notificar cuidadores em caso de não adesão
- Gerar relatórios de acompanhamento de adesão

O software permitirá ao usuário **cadastrar pacientes, cadastrar médicos, cadastrar cuidadores, registrar prescrições, agendar lembretes de medicação automaticamente via prescrição, registrar tomada de medicamentos e visualizar relatórios de adesão.**

Escopo Negativo: Este sistema não realizará agendamento de consultas, registro de médicos no aplicativo (app mobile), busca por hospitais e unidades de saúde, ou processamento de folhas de pagamento.

1.2 Requisitos Funcionais

[RF01] - Cadastro de Medicamentos

- **Nome do Caso de Uso:** Cadastrar Medicamento
- **Breve descrição:** Permitir que médicos cadastrem medicamentos associados a uma prescrição médica. O sistema deve permitir incluir nome, dosagem, frequência e horário de tomada.
- **Atores envolvidos:** Médico
- **Pré-condições:** Médico estar autenticado no sistema; paciente estar cadastrado
- **Fluxo Principal de Eventos:**
 1. Médico acessa a funcionalidade de prescrição
 2. Seleciona ou cria uma prescrição para o paciente

3. Adiciona medicamento(s) com: nome, dosagem, frequência (vezes ao dia), horários específicos, quantidade de doses total
 4. Sistema valida os dados
 5. Sistema armazena o medicamento vinculado à prescrição
- **Pós-condições:** Medicamento vinculado à prescrição do paciente; agendamento criado automaticamente para os horários definidos
 - **Fluxo Secundário de Eventos:**
 - Médico edita medicamento já cadastrado
 - Médico remove medicamento da prescrição (soft delete)
 - **Observações:** Medicamento é atrelado a uma prescrição, não existe isoladamente. O agendamento das doses é criado automaticamente pelo sistema.

[RF02] - Cadastro de Pacientes

- **Nome do Caso de Uso:** Cadastrar Paciente
- **Breve descrição:** Permitir o cadastro de pacientes idosos no sistema. Cada paciente pode ser vinculado a um ou mais cuidadores que monitoram sua medicação.
- **Atores envolvidos:** Cuidador, Administrador do Sistema
- **Pré-condições:** -
- **Fluxo Principal de Eventos:**
 1. Usuário acessa a funcionalidade de cadastro de paciente
 2. Informa dados do paciente: nome, data de nascimento, contato, histórico de saúde (opcional)
 3. Sistema valida os dados
 4. Sistema armazena o paciente
- **Pós-condições:** Paciente cadastrado e disponível para receber prescrições
- **Fluxo Secundário de Eventos:**
 - Cuidador é vinculado ao paciente
 - Dados do paciente são editados
 - Paciente é desativado
- **Observações:** Paciente é o usuário final que toma os medicamentos. O monitoramento é feito pelos cuidadores vinculados.

[RF03] - Cadastro de Médicos

- **Nome do Caso de Uso:** Cadastrar Médico
- **Breve descrição:** Permitir o cadastro de médicos no sistema para que possam criar e gerenciar prescrições de medicamentos para seus pacientes.
- **Atores envolvidos:** Médico, Administrador do Sistema

- **Pré-condições:** -
- **Fluxo Principal de Eventos:**
 1. Médico acessa a funcionalidade de cadastro
 2. Informa dados: nome, CRM, especialidade, contato
 3. Sistema valida os dados
 4. Sistema armazena o médico
- **Pós-condições:** Médico cadastrado e capaz de criar prescrições
- **Fluxo Secundário de Eventos:**
 - Dados do médico são editados
 - Médico é desativado
- **Observações:** Médico é o ator que prescreve medicamentos aos pacientes.

[RF04] - Cadastro de Cuidadores

- **Nome do Caso de Uso:** Cadastrar Cuidador
- **Breve descrição:** Permitir o cadastro de familiares/cuidadores que monitoram a medicação de pacientes idosos. Cuidadores são vinculados a pacientes via convite.
- **Atores envolvidos:** Cuidador, Paciente
- **Pré-condições:** Paciente estar cadastrado
- **Fluxo Principal de Eventos:**
 1. Sistema gera código/token de convite para o paciente
 2. Cuidador recebe o convite
 3. Cuidador aceita o convite e se cadastra no sistema
 4. Sistema vincula o cuidador ao paciente
- **Pós-condições:** Cuidador vinculado ao paciente e capaz de receber notificações sobre a medicação
- **Fluxo Secundário de Eventos:**
 - Cuidador rejeita o convite
 - Cuidador é desvinculado do paciente
 - Cuidador atualiza seus dados
- **Observações:** Um paciente pode ter múltiplos cuidadores vinculados. Cuidadores recebem notificações quando o paciente não toma a medicação.

[RF05] - Agendamento Automático de Notificações

- **Nome do Caso de Uso:** Agendar Notificações Automaticamente
- **Breve descrição:** Quando uma prescrição é criada ou ativada, o sistema cria automaticamente os agendamentos de notificação com base nos horários e frequência definidos na prescrição.

- **Atores envolvidos:** Sistema (processo automático)
- **Pré-condições:** Prescrição criada com medicamentos e horários definidos
- **Fluxo Principal de Eventos:**
 1. Prescrição é criada ou ativada
 2. Sistema extrai horários e frequência de cada medicamento
 3. Sistema cria agendamentos de notificação para cada dose
- **Pós-condições:** Todos os agendamentos de notificação criados e aguardando execução
- **Fluxo Secundário de Eventos:**
 - Prescrição é editada - agendamentos antigos são cancelados e novos são criados
 - Prescrição é desativada ou cancelada - todos os agendamentos são cancelados
- **Observações:** Não existe um cadastro separado de regras - o sistema deriva os agendamentos automaticamente a partir da prescrição.

[RF06] - Envio de Lembretes de Medicamentos

- **Nome do Caso de Uso:** Enviar Lembretes de Medicamentos
- **Breve descrição:** Quando um agendamento atinge o horário definido, o sistema envia uma notificação ao paciente e aos cuidadores vinculados lembrando da toma do medicamento.
- **Atores envolvidos:** Sistema (processo automático)
- **Pré-condições:** Agendamento existir e estar ativo
- **Fluxo Principal de Eventos:**
 1. Agendamento atinge horário de notificação
 2. Sistema identifica paciente e cuidadores vinculados
 3. Sistema envia notificação *push* para o dispositivo do paciente
 4. Sistema envia notificação *push* para todos os cuidadores vinculados
- **Pós-condições:** Notificação enviada; registro de notificação criado
- **Fluxo Secundário de Eventos:**
 - Notificação *push* falha - sistema tenta novamente com *backoff* exponencial
 - Notificação continua falhando após máximo de tentativas - mensagem vai para *dead letter queue*
- **Observações:** Notificações são enviadas via *Firebase Cloud Messaging (FCM)*. O sistema deve garantir que falhas em cuidadores sejam tratadas com *retry*.

[RF07] - Registro de Uso de Medicamentos

- **Nome do Caso de Uso:** Registrar Uso de Medicamento

- **Breve descrição:** Permitir que o paciente (ou cuidador) registre se a dose do medicamento foi tomada ou perdida no horário agendado.
- **Atores envolvidos:** Paciente, Cuidador
- **Pré-condições:** Agendamento existir e estar no horário de tomada
- **Fluxo Principal de Eventos:**
 1. Paciente ou cuidador recebe notificação de lembrete
 2. Usuário confirma que tomou o medicamento (confirmar dose) ou indica que perdeu (registrar falha)
 3. Sistema atualiza o registro de dose com o status correspondente
- **Pós-condições:** Registro de dose atualizado com status de tomado ou perdido
- **Fluxo Secundário de Eventos:**
 - Cuidador registra toma em nome do paciente
 - Usuário não interage - sistema marca automaticamente como perdido após janela de tolerância
- **Observações:** O registro de dose é fundamental para calcular o relatório de adesão (RF08).

[RF08] - Relatórios de Adesão

- **Nome do Caso de Uso:** Gerar Relatório de Adesão
- **Breve descrição:** Permitir que cuidadores visualizem o percentual de adesão do paciente à medicação em um período definido, baseado nos registros de doses tomadas e perdidas.
- **Atores envolvidos:** Cuidador, Médico
- **Pré-condições:** Registros de doses existirem
- **Fluxo Principal de Eventos:**
 1. Cuidador solicita relatório de adesão para um paciente
 2. Sistema define período do relatório (ex: últimos 30 dias)
 3. Sistema calcula percentual com base nos registros: $(\text{doses tomadas} / \text{total de doses}) * 100$
 4. Sistema apresenta relatório com métricas de adesão
- **Pós-condições:** Relatório de adesão apresentado ao solicitante
- **Fluxo Secundário de Eventos:**
 - Relatório é exportado em formato PDF
 - Histórico de relatórios anteriores é disponível
- **Observações:** Relatório deve ser capaz de identificar padrões de não-adesão (horários específicos, dias da semana).

[RF09] - Notificar Médicos sobre Adesão

- **Nome do Caso de Uso:** Notificar Médico sobre Adesão
- **Breve descrição:** Quando um paciente apresenta baixa adesão à medicação (abaixo de um limiar definido), o sistema deve notificar o médico que prescreveu a medicação.
- **Atores envolvidos:** Sistema (processo automático)
- **Pré-condições:** Relatório de adesão calculado; médico estar vinculado à prescrição
- **Fluxo Principal de Eventos:**
 1. Sistema identifica paciente com adesão abaixo do limiar (ex: 80)
 2. Sistema identifica médico que criou a prescrição
 3. Sistema envia notificação ao médico sobre a baixa adesão
- **Pós-condições:** Médico notificado sobre baixa adesão do paciente
- **Fluxo Secundário de Eventos:**
 - Médico recebe notificação com link para ver relatório detalhado
 - Médico pode entrar em contato com o paciente ou cuidador
- **Observações:** Notificação ao médico é uma medida de segurança para intervenção precoce em caso de não-adesão.

1.3 Requisitos Não-Funcionais

[RNF01] - Disponibilidade 24/7

- **Categoria:** Requisitos de confiabilidade
- **Descrição:** O sistema backend deve estar disponível 24 horas por dia, 7 dias por semana, para garantir que notificações sejam enviadas pontualmente. Tempo de disponibilidade mínimo: 99% do tempo.
- **Justificativa:** Pacientes dependem de lembretes de medicação em horários específicos. Falhas no sistema podem resultar em perda de doses.

[RNF02] - Acessibilidade Mobile

- **Categoria:** Requisitos de sistema
- **Descrição:** O sistema deve ser acessível via dispositivos móveis (iOS e Android) através do aplicativo CareConnect, permitindo que pacientes e cuidadores acessem suas funcionalidades principais.
- **Justificativa:** O público-alvo principal (idosos e cuidadores) utiliza predominantemente dispositivos móveis para interação com o sistema.

[RNF03] - Tempo de Resposta

- **Categoria:** Requisitos de eficiência
- **Descrição:** O sistema deve processar requisições da API em menos de 5 segundos em condições normais de operação, garantindo resposta ágil para ações de confirmação de dose e consulta de prescrições.

- **Justificativa:** Usuários esperam feedback imediato ao confirmar uma dose ou consultar seu cronograma de medicamentos.

[RNF04] - Armazenamento Seguro de Dados

- **Categoria:** Requisitos de segurança
- **Descrição:** Dados dos usuários devem ser armazenados de forma segura no banco de dados, utilizando práticas de criptografia e controle de acesso. Dados sensíveis (saúde, medicação) devem ter proteção reforçada.
- **Justificativa:** O sistema lida com dados pessoais e de saúde sensíveis - proteção é requisito legal e ético.

[RNF05] - Conformidade com LGPD

- **Categoria:** Requisitos legais
- **Descrição:** O sistema deve operar em conformidade com a Lei Geral de Proteção de Dados (LGPD). Isso inclui: minimização da coleta de dados, direito ao acesso e apagamento dos dados do usuário, e política de retenção de dados.
- **Justificativa:** Dados de saúde são considerados dados sensíveis pela LGPD e exigem base legal específica para tratamento.

[RNF06] - Portabilidade do Sistema

- **Categoria:** Requisitos de portabilidade
- **Descrição:** O sistema backend deve ser desenvolvido para funcionar em diferentes ambientes (desenvolvimento, staging, produção) sem alterações de código, utilizando variáveis de ambiente para configuração.
- **Justificativa:** Facilita o deploy e a manutenção em diferentes infraestrutura.

[RNF07] - Interoperabilidade com Sistemas Externos

- **Categoria:** Requisitos de interoperabilidade
- **Descrição:** O sistema deve ser capaz de integrar com sistemas externos de prescrição digital (ex.: Memed) e plataformas de notificação (ex.: Firebase Cloud Messaging).
- **Justificativa:** A integração com plataformas de prescrição digital é parte do escopo do CareConnect para importação automática de receitas.

[RNF08] - Cobertura de Testes

- **Categoria:** Requisitos de implementação
- **Descrição:** O sistema deve possuir testes unitários com cobertura mínima de 70% do código. Testes de integração devem cobrir os principais fluxos de funcionamento.
- **Justificativa:** Garantia de qualidade e redução de regressões em um sistema crítico de saúde.

1.4 Protótipo

O CareConnect é um sistema centrado em notificações e agendamento automático de medicação. Sua proposta de valor principal é a pontualidade e confiabilidade no envio de lembretes, tornando a interação do usuário com o sistema relativamente simples e transparente.

Como o sistema não requer entrada de dados complexa pelo usuário final (paciente/cuidador) - o fluxo principal é a visualização do cronograma e a confirmação de dose - optou-se por não desenvolver protótipos mockup tradicionais. O aplicativo mobile está em desenvolvimento e pode ser consultado para referência visual.

Arquitetura de Integração de Prescrições:

O fluxo de prescrições do CareConnect funciona da seguinte forma:

1. Médico cria prescrição em plataforma de prescrição digital (ex.: Memed)
2. Plataforma externa envia prescrição ao backend CareConnect via API REST
3. Backend extrai medicamentos, horários e frequência da prescrição
4. Sistema cria automaticamente os agendamentos de notificação
5. Paciente e cuidadores recebem notificações nos horários programados

Funcionalidades principais acessíveis ao usuário:

- **Paciente/Cuidador:** Visualizar cronograma de medicamentos, confirmar dose tomada, visualizar relatórios de adesão
- **Médico:** Cadastrar e gerenciar prescrições (via integração API ou manualmente), receber alertas de baixa adesão
- **Sistema:** Enviar notificações, registrar doses, gerar relatórios de adesão (processos automáticos)

Sobre a ausência de protótipos: Os protótipos visuais do aplicativo mobile estão em desenvolvimento no repositório care-connect (React Native). **TODO: link para o repo do app mobile**

2 Projeto de Software

Texto Base (Instrução):

Esta seção descreve o design (o "como") do software, traduzindo os requisitos (o "o quê") em uma especificação de implementação. Esta seção é voltada para os desenvolvedores.

2.1 Diagramas de Interação

Texto Base (Instrução):

Insira aqui os Diagramas de Interação da UML (Diagramas de Sequência detalhados ou Diagramas de Comunicação). Diferente do DSS (3.1.4), estes diagramas mostram a colaboração interna entre as classes e objetos de software (ex: Controladores, Repositórios, Entidades) para realizar um Caso de Uso.

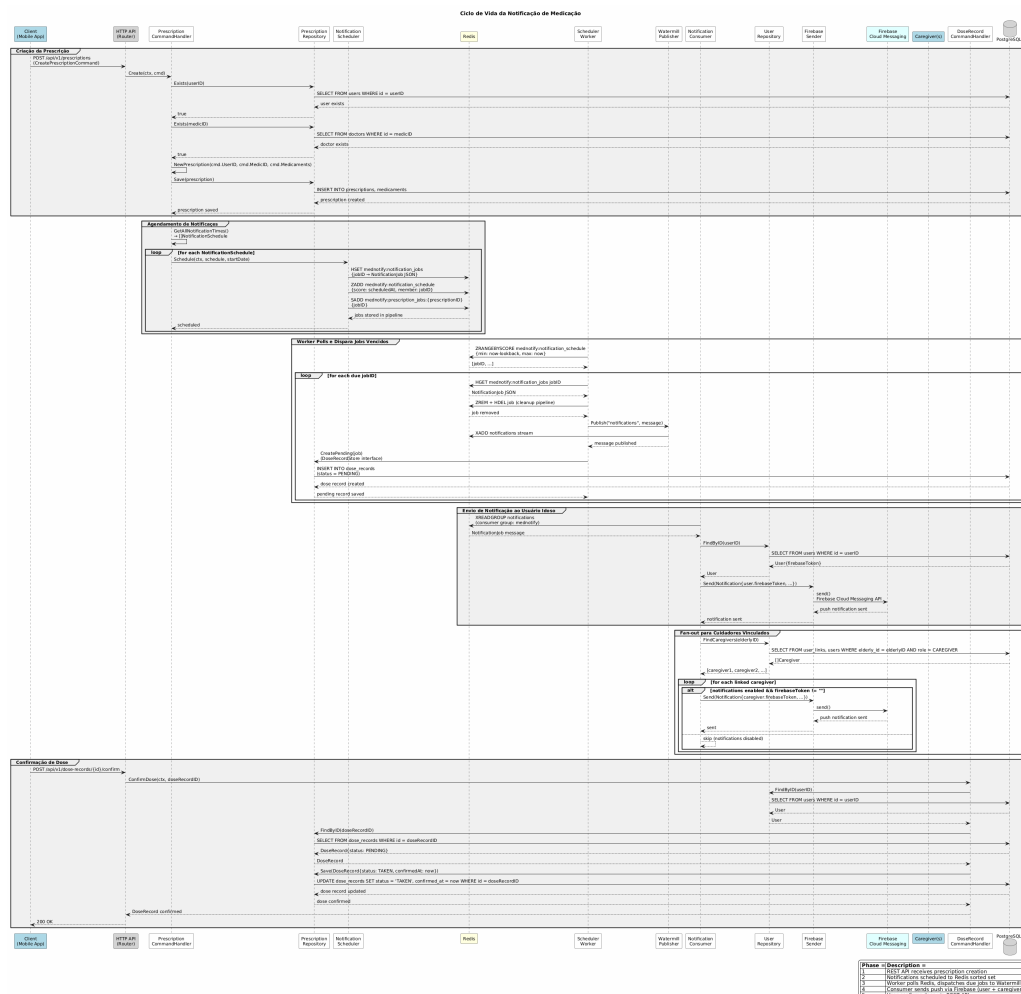


Figure 1: Diagrama de Sequência.

2.2 Diagramas de Classes

Texto Base (Instrução):

Insira aqui o Diagrama de Classes de Projeto (UML). Este diagrama é mais detalhado que o Modelo Conceitual (3.2). Ele deve incluir as classes de software (ex: classes de Interface, Controle e Entidade), seus atributos (com tipos de dados e visibilidade -/+) e seus métodos (com parâmetros e visibilidade).

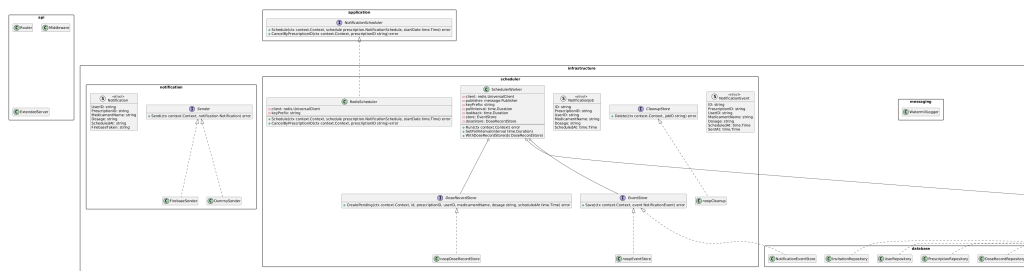


Figure 2: Diagrama de Classes.

2.3 Persistência de Dados

O banco de dados utilizado no projeto é o **PostgreSQL**, classificado como **relacional**. A escolha se deve ao **banco de dados ser muito atual e confiável, com um ótimo desempenho, escalabilidade, compatibilidade tecnológica ao nosso projeto graças a linguagem estruturada (sql)**.

Para bancos de dados relacionais, a persistência dos dados é realizada por meio de mapeamento objeto-relacional (ORM), no qual as classes do Diagrama de Classes são mapeadas para tabelas, e seus atributos para colunas, respeitando chaves primárias, estrangeiras e relacionamentos.

2.4 Mapeamento de Ferramentas

Texto Base (Instrução):

Descreva as ferramentas utilizadas no desenvolvimento, versionamento, testes e deploy do sistema, justificando brevemente a escolha de cada uma em relação aos requisitos do projeto, como escalabilidade, manutenção, produtividade e integração.

2.4.1 Linguagem de Programação

A linguagem **Go**, **Typescript** e **Javascript** foi adotada neste projeto por apresentar melhor aderência às necessidades da aplicação, especialmente no que se refere a **aplicativos Mobile** (back-end) e design de front-end. Entre os principais motivos para sua escolha destacam-se:

- Produtividade, tipagem, performance, comunidade
- Facilidade, documentação, segurança
- Vasta gama de bibliotecas com muitas funcionalidades e utilidades para diversos problemas

2.4.2 Frameworks e Bibliotecas

O framework **React Native** foi utilizado para estruturar a aplicação, pois oferece **padrão arquitetural, segurança, rapidez no desenvolvimento, suporte a ORM, middlewares e desenvolvimento mobile**. Outras bibliotecas relevantes incluem:

- Swagger API: utilizada para **Desenvolvimento de API's**
- Memed API: utilizada para **Dados de medicamentos**
- Twilio API: utilizada para **Envio de notificações**
- NativeWind (Tailwind CSS): utilizada para **Framework de estilização CSS**

- Expo Notifications: utilizada para **Notificações**
- Zustand + AsyncStorage: utilizada para **Armazenamento local**

2.4.3 Ferramentas de Desenvolvimento

Para o desenvolvimento e apoio ao projeto, foram utilizadas as seguintes ferramentas:

- IDE/Editor – [**VS Code**]: para edição e organização do código-fonte
- Ferramenta de modelagem – [**PlantUML**]: para criação de diagramas UML
- Ferramenta de testes de API – [**Postman**]: para validação das rotas e serviços
- Ferramenta de mock/teste – [**Docker**]: para simulação de serviços externos (quando aplicável)

2.4.4 Containerização e Automação

A ferramenta **Docker** foi utilizada para padronizar o ambiente de desenvolvimento e facilitar o deploy da aplicação, garantindo que ela funcione de forma consistente em diferentes ambientes.

3 Versionamento

3.1 Introdução

Nesse guia padronizaremos a forma como o time desenvolve, integra e entrega software. Usaremos o **Git Flow** para organizar as ramificações e o **Versionamento Semântico** para comunicar o impacto das mudanças.

3.2 Configuração do Ambiente

Teremos um link (a ser criado), com os devs adicionados como colaboradores, onde serão criados repositórios específicos para cada projeto.

3.3 Versionamento Semântico (SemVer)

Vamos usar o padrão **Major.Minor.Patch** (ex: v1.2.0). A alteração dos números depende do impacto das mudanças feitas no código:¹

3.3.1 Estrutura da Versão (X.Y.Z)

- **MAJOR (X.0.0) - Quebra de Compatibilidade:** Incrementada quando há mudanças drásticas. É geralmente quando acaba a retrocompatibilidade.
- **MINOR (0.Y.0) - Nova Funcionalidade:** Incrementada quando tem novas funcionalidades adicionadas, mas que ainda são compatíveis com versões anteriores. Exemplo: Adicionar um novo botão na interface.
- **PATCH (0.0.Z) - Correção de Bug:** Incrementada para correções de falhas simples que não alteram funcionalidades. Exemplo: Corrigir um erro de digitação, ajustar uma cor CSS ou corrigir um cálculo.

3.4 O Modelo Git Flow

A estrutura do repositório vai ser composta pelas seguintes branches principais:²

3.4.1 Branches Permanentes

- **main:** Representa a **Produção**. **Não recebe commit direto**. Só recebe código via Merge de **release** ou **hotfix**. Cada commit deve ter uma **Tag** de versão.
- **develop:** Representa o **Desenvolvimento Contínuo**, é a branch de integração. Ela **contém as funcionalidades completas para a próxima versão**.

3.4.2 Feature Branches (feature/*)

- **Objetivo:** Desenvolver uma nova funcionalidade.
- **Nasce em:** develop.
- **Morre em:** develop.

¹Se quiserem ler mais sobre versionamento semântico podem acessar a [especificação](#).

²Se quiserem ler mais sobre o modelo do Git Flow, podem encontrar o artigo original do Vincent Driessen (2010), "[A successful Git branching model](#)" ou esse em [português](#).

- **Fluxo:** O dev cria a branch, trabalha nela e abre um PR para a **develop**. Após o merge, a branch local pode ser apagada.

3.4.3 Release Branches (release/*)

- **Objetivo:** Congela o código para testes de QA e preparação final (documentação, versão), aqui acontece o **Staging**. **Ela é exclusiva para isso, nenhuma feature nova entra aqui.**
- **Nasce em:** **develop** (quando o time decide que vai lançar uma versão).
- **Morre em:** Dois lugares. Ao finalizar a release, ela é mergeada na:
 1. **main:** Para atualizar a produção.
 2. **develop:** Para garantir que correções de bugs feitas durante a fase de release voltem para o desenvolvimento.
- O nome da branch deve seguir o SemVer (ex: **release/v1.2.0**).

3.4.4 Hotfix Branches (hotfix/*)

- **Objetivo:** Resolver bugs críticos em produção.
- **Nasce em:** **main**.
- **Morre em:** Assim como a release, ela é mergeada na:
 1. **main:** Para corrigir o erro imediatamente (gera nova Tag Patch).
 2. **develop:** Para garantir que o erro não volte a aparecer na próxima release.
- Geralmente incrementa o **Patch** (ex: **hotfix/v1.2.1**).

3.5 Exemplo:

Por exemplo, imagine que estamos na **versão v1.1.0**.

1. **Início do Trabalho:** O dev quer criar um "Modo Escuro". Ele cria a branch **feature/dark-mode** a partir da **develop**.
2. **Integração:** Ele termina, abre PR e mergeia na **develop**. Outros devs também mergeiam suas features.
3. **Corte da Release:** O time decide lançar. É criada a branch **release/v1.2.0** a partir da **develop**.
4. **Fase de QA:** O QA testa a **release/v1.2.0**. Encontra um bug no CSS.
5. **Correção na Release:** O dev corrige o bug na branch **release/v1.2.0** (commit de fix).
6. **Lançamento:** A release é aprovada.
 - Mergeiam a **release/v1.2.0** na **main** → Cria-se a Tag **v1.2.0**.
 - Mergeiam a **release/v1.2.0** na **develop** (o bug é corrigido na **develop** também).
7. **Hotfix:** No dia seguinte, descobrem que o login parou de funcionar na produção (**main**).

8. Correção do Hotfix:

- Criam hotfix/v1.2.1 a partir da main.
- Corrigem o erro.
- Mergeiam na main (Tag v1.2.1) e na develop.

3.6 Glossário de Comandos

Aqui temos um glossário dos comandos que mais usaremos no git, caso alguém não se lembre ou não esteja acostumado. Pode também rodar o comando `git --help` direto no terminal, acessar a [documentação do Git](#) ou o [glossário da Atlassian](#).

3.6.1 Configuração Inicial

```
1 git clone https://github.com/repositorioasercariado/seu-projeto.git
2 git clone https://github.com/repositoioasercariado/seu-projeto.git
3 cd seu-projeto
```

3.6.2 Operações Diárias

Fluxo Básico para Feature:

```
1 # 1. Garanta que esta atualizado
2 git checkout develop
3 git pull origin develop
4
5 # 2. Crie sua branch
6 git checkout -b feature/minha-tarefa
7
8 # ... Trabalho sendo feito ...
9
10 # 3. Salve e envie
11 git add .
12 git commit -m "feat: Adiciona nova tela"
13 git push origin feature/minha-tarefa
```

3.6.3 Boas Práticas de Commit

Para que seja fácil entender e encontrar o que desejamos, é bom seguir padrões de commit:

- **feat:** Nova funcionalidade.
- **fix:** Correção de bug.
- **docs:** Alteração em documentação.
- **style:** Formatação (ponto e vírgula, espaços).
- **refactor:** Melhoria de código sem mudar funcionalidade.

References

- [1] *Integração de software..* Disponível em: <https://apipass.com.br/integracao-de-software-como-funciona/>
- [2] *Guia completo sobre Integração de Software..* Disponível em: <https://www.techverdi.com/pt/blog>
- [3] *Enterprise Integration Patterns..* Disponível em: <https://www.enterpriseintegrationpatterns.com/>
- [4] *iPaaS vs ESB. .* Disponível em: <https://latenode.com/pt-br>